

TPY1101



WebLandingSuite

Integrantes:

Juan Carlos Agüero Cárcamo

Marcos Orellana

Isaac González

Plataforma Web Inteligente para la Creación Automatizada de Landing Pages

App desplegada en la nube · Demostración funcional en vivo

React 19

Spring Boot

FastAPI

PostgreSQL

Vercel + Render

Punto de Partida — Resumen Ejecutivo

Contexto y Problema Central

Microempresarios y freelancers no pueden crear landing pages profesionales: las herramientas existentes son complejas, costosas o ambas. La IA democratiza este acceso.

Mercado Objetivo

Emprendedor/Freelancer (25–50 años, sin conocimientos técnicos, presupuesto limitado). Usuario 2: Gestor de marketing / agencia pequeña.

Objetivo General

Desarrollar una plataforma SaaS con IA que automatice la maquetación de landing pages en 3 niveles de calidad (Básico, Intermedio, Premium), con pago único y descarga de código.

Objetivos Específicos

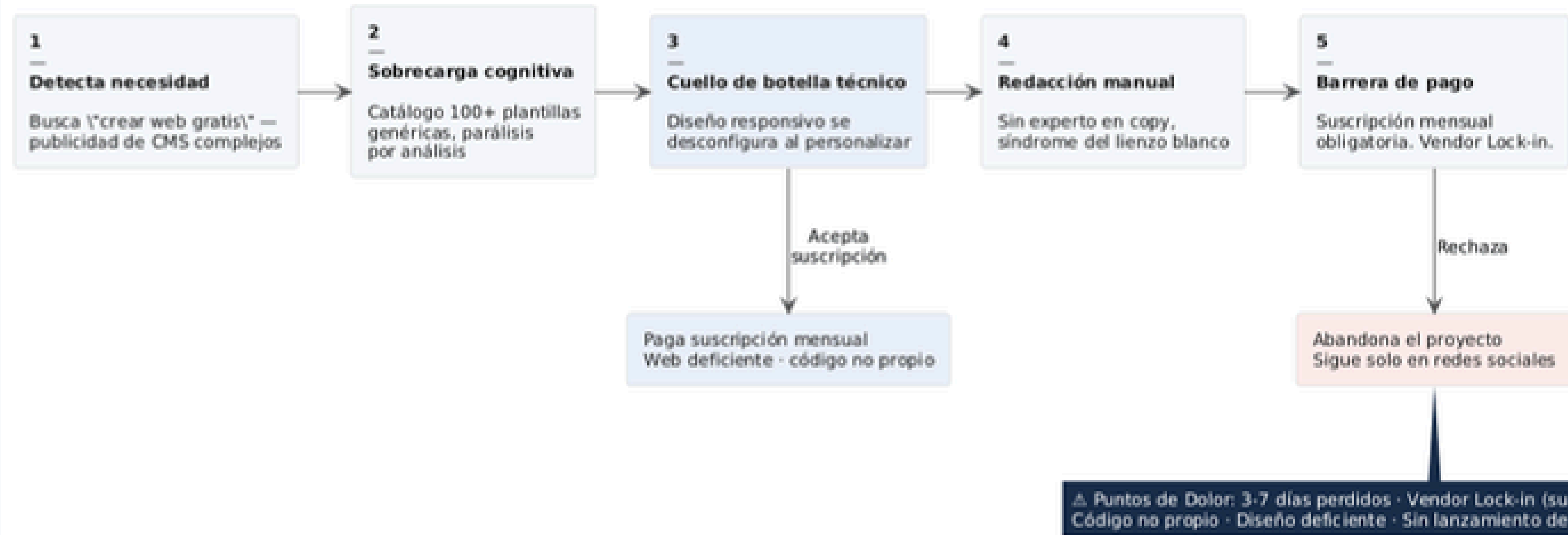
- 1) BD relacional + backend multicapa.
- 2) Motor de Prompt Engineering escalonado.
- 3) Frontend React guiado en < 10 pasos.
- 4) Post-procesamiento y entrega de archivos listos.



Arbol del problema



Situación Inicial del Cliente — Proceso Actual (AS-IS)



Planificación del Proyecto — Hitos y Carta Gantt

#	Hito	Fecha	Estado
H-01	Kick-off del Proyecto	14/Abr/2026	Completado
H-03	Entornos Configurados (Neon + Render)	28/Mar/2026	Completado
H-04	Backend Core Operativo (JWT + Swagger)	22/Abr/2026	Completado
H-05	Microservicio IA Integrado (FastAPI)	24/Abr/2026	Completado
H-06	Frontend Funcional (React + flujo completo)	14/May/2026	Completado
H-07	Integración Full-Stack Validada	28/Abr/2026	Completado
H-08	MVP en Producción (Vercel + Render)	14/May/2026	Completado
H-09	Entrega EA2	25/May/2026	Completado

Sprint Actual

44

tareastotales

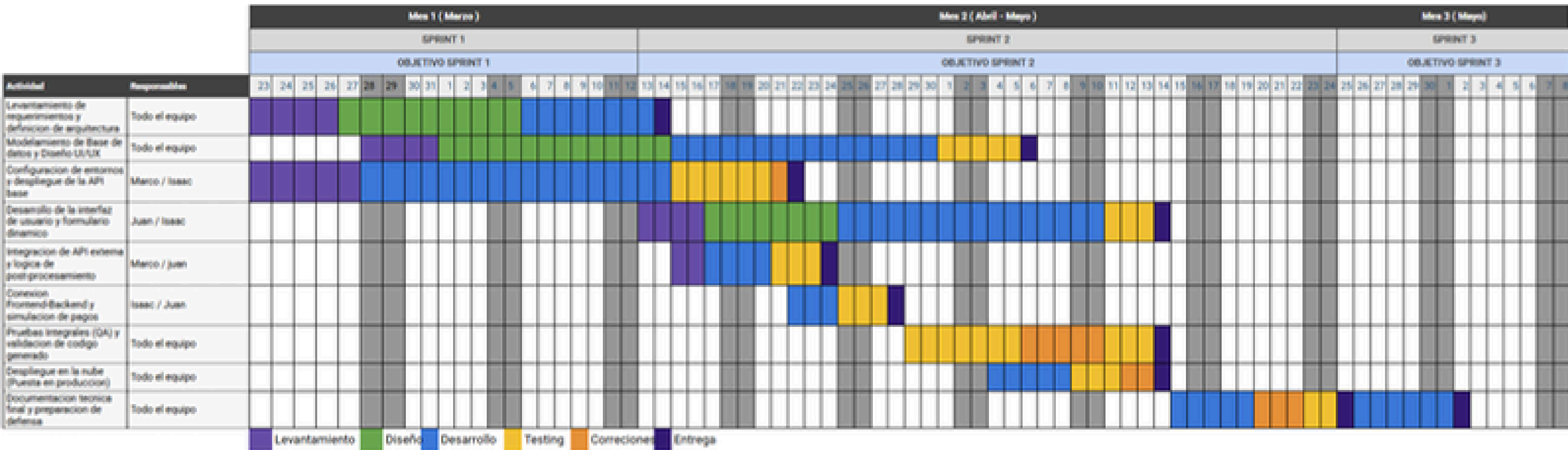
44

Finalizadas



Carta Gantt

Plataforma Web Inteligente para la Creacion Automatizada de Landing Page



Servicios Cloud Utilizados

PaaS

Vercel

Frontend

SPA React/Vite distribuida por CDN global. CI/CD automático desde rama main. SSL incluido.

PaaS

Render

Backend + IA

Web Service Java 21 (Spring Boot) y microservicio Python 3 (FastAPI) desplegados como servicios independientes.

DBaaS

Neon

Base de Datos

PostgreSQL serverless. Backups automáticos. Accesible exclusivamente desde el backend via variables de entorno.

SaaS

OpenRouter / OpenAI

Motor IA

API externa que procesa prompts escalonados (Básico/Intermedio/Premium) y retorna HTML/CSS válido.

SaaS

UptimeRobot

Monitoreo

Ping constante a endpoints críticos. Alertas inmediatas al equipo ante caídas o latencia elevada.

SaaS

Jira + GitHub

Gestión / CI

Tablero Kanban con 44 tareas. Git Flow con PRs y Code Review obligatorio antes del merge a main.



Solución TO-BE — WebLandingSuite



F1 Formulario Dinámico de Captura

React con validaciones en tiempo real. Guía al usuario en < 10 pasos para definir nombre, rubro, colores y estilo de comunicación.

F2 Motor Prompt Engineering Escalonado

Backend Spring Boot que selecciona automáticamente la plantilla de instrucciones (Básico / Intermedio / Premium) e inyecta los datos del cliente.

F3 Post-procesamiento y Sanitización

Algoritmo que recibe el HTML/CSS crudo de la IA, elimina texto conversacional, valida el DOM y previene XSS.

F4 Live Preview + Descarga .zip

Iframe aislado para previsualización segura. Entrega final empaquetada en .zip con HTML/CSS/JS listos para hosting.

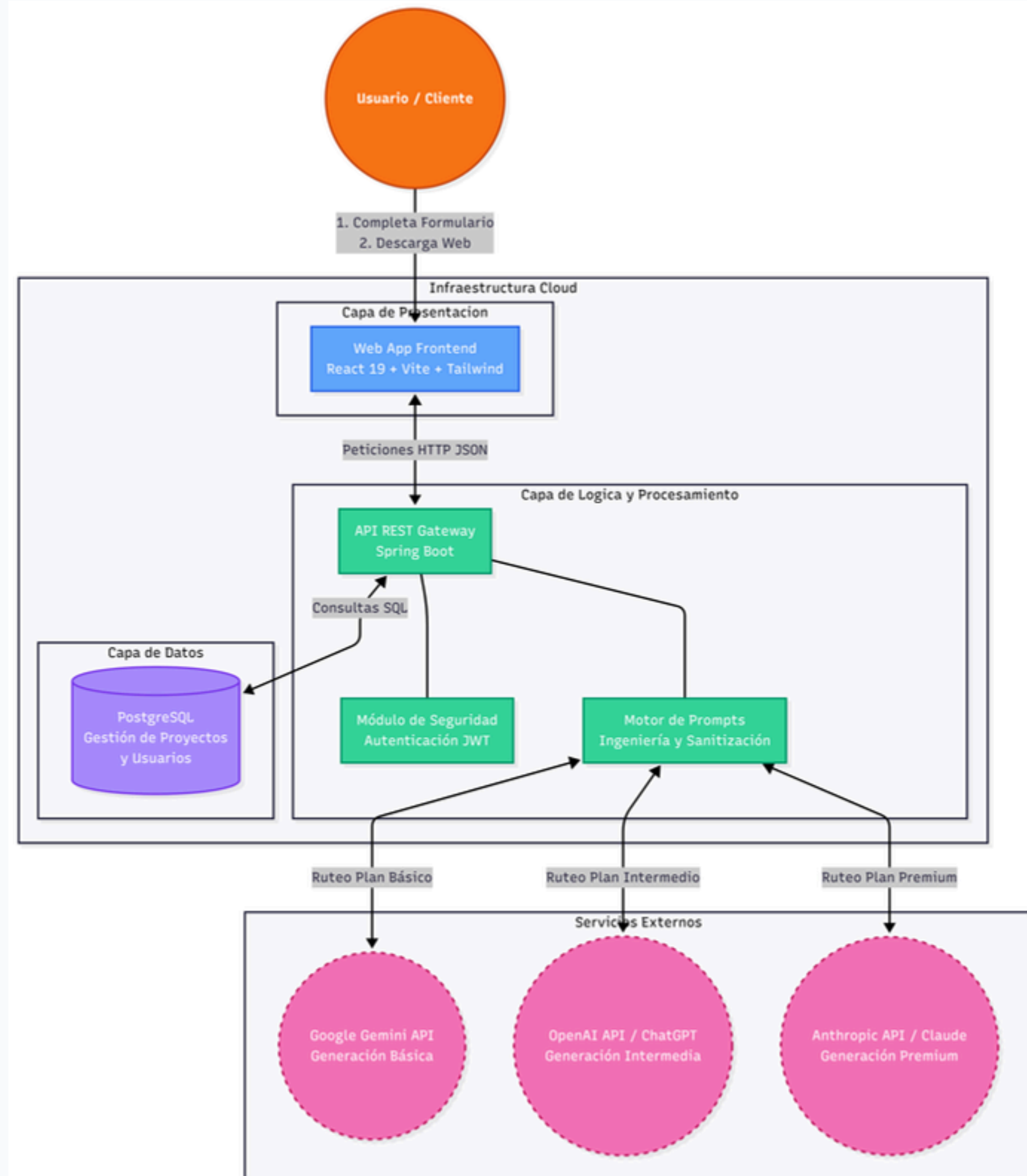
F5 Dashboard de Proyectos

Panel privado con historial de landing pages generadas, acceso a descargas previas y registro de transacciones simuladas.

F6 Seguridad JWT + Auditoría

Autenticación stateless JWT. Tabla de logs que registra eventos críticos (login, creación, errores) con IP y timestamp.

Diagrama de Arquitectura



Alcance, Supuestos y Restricciones

✓ INCLUYE

- Registro, autenticación JWT y gestión de perfiles
- Formulario dinámico inteligente (< 10 pasos)
- Motor Prompt Engineering en 3 niveles (Básico / Intermedio / Premium)
- Post-procesamiento: limpieza y validación de HTML/CSS/JS generado
- Live Preview en iframe aislado y seguro (anti-XSS)
- Simulación de facturación por plan seleccionado
- Descarga de archivos .zip listos para hosting
- Panel Dashboard con historial de proyectos

✗ FUERA DE ALCANCE

- Hosting de los sitios generados para el cliente final
- Gestor de contenidos post-descarga (editor de la web generada)
- Pasarelas de pago reales (Webpay, Stripe, PayPal)
- Generación de tiendas online con carrito de compras
- Configuración o compra de dominios personalizados (.com, .cl)
- Sistemas web complejos dentro de la landing generada



Metodología Híbrida — Cascada + Scrum Adaptado

Metodología Elegida

Fase 1 — Cascada: Análisis, arquitectura, modelo de datos y stack. Decisiones estables antes de escribir código.

Fase 2 — Scrum Adaptado: Sprint único de 44 tareas gestionadas en Jira (Por Hacer → En Curso → En Revisión → Finalizado).

Git Flow: Ramas feature → dev → main. Pull Request + Code Review obligatorio antes de cada merge.

DoD: Código funcional, validaciones activas, PR aprobado, sin regresiones, JWT en endpoints protegidos.

Roles del Equipo



Juan Carlos Agüero

Frontend / UI · React, Atomic Design, integración API



Marcos Orellana

Backend / BD · Spring Boot, PostgreSQL, JWT



Isaac González

Ingeniero IA · FastAPI Python, prompts escalonados, OpenRouter

Tablero en Jira

Resumen Backlog Tablero Código Cronograma Documentos Formularios Desarrollo

Buscar en el ba... Filtros

SCRUM Sprint 6 14 abr - 28 abr (45 actividades)

Completar sprint

600uAM-49 Rediseñar pagina Home (v2.0)	FINALIZADO	✓	👤
600uAM-48 Implementar pagina de Nosotros	FINALIZADO	✓	👤
600uAM-47 Refactorizar Footer Global con nueva UI	FINALIZADO	✓	👤
600uAM-37 Validación de formularios en frontend	FINALIZADO	✓	👤
600uAM-34 Pensar Roles y Beneficios	FINALIZADO	✓	👤
600uAM-33 Implementación de manejo de estados y loading	FINALIZADO	✓	👤
600uAM-30 Diseño responsivo de la aplicación	FINALIZADO	✓	👤
600uAM-40 Implementación de autenticación en frontend	FINALIZADO	✓	👤
600uAM-24 Pensar IA para utilización en el Backend	FINALIZADO	✓	👤
600uAM-42 implementación de pagina de contacto con Atomic Design y validaciones	FINALIZADO	✓	👤
600uAM-26 Crear el Servicio de Generación con IA (AiService)	FINALIZADO	✓	👤
600uAM-31 Pensar diseño Frontend (Logo, paleta de colores, footer, etc)	FINALIZADO	✓	👤
600uAM-36 Integración de API de backend en frontend	FINALIZADO	✓	👤
600uAM-29 Integrar la IA en el flujo de Creación de Proyectos (LandingProjectService)	FINALIZADO	✓	👤
600uAM-27 Investigar y obtener API Key del proveedor de IA	FINALIZADO	✓	👤
600uAM-38 Validación de entradas DTOs	FINALIZADO	✓	👤
600uAM-28 Configurar políticas de CORS	FINALIZADO	✓	👤
600uAM-35 Desarrollo de servicio de IA mediante FastAPI en Python	FINALIZADO	✓	👤
600uAM-5 Creación del repositorio-backend Github	FINALIZADO	✓	👤
600uAM-39 Configuración inicial del entorno Frontend usando React + Vite	FINALIZADO	✓	👤
600uAM-23 Implementación de Docker	FINALIZADO	✓	👤
600uAM-7 Definir la estructura de paquetes (carpetas)(services, models, repositories, dto y config)	FINALIZADO	✓	👤
600uAM-32 Ocultar la contraseña BD properties (Ocupar Variables de Entorno)	FINALIZADO	✓	👤

Historias de Usuario

HU01: Como visitante, quiero navegar por la Landing del producto para entender qué ofrece.

HU02: Como visitante, quiero registrarme con mi correo y contraseña para crear mi cuenta.

HU12: Como usuario, quiero acceder a mi Dashboard para ver una lista con todos mis proyectos creados.

HU05: Como administrador, quiero gestionar (crear/editar/eliminar) los planes de diseño para actualizar la oferta.

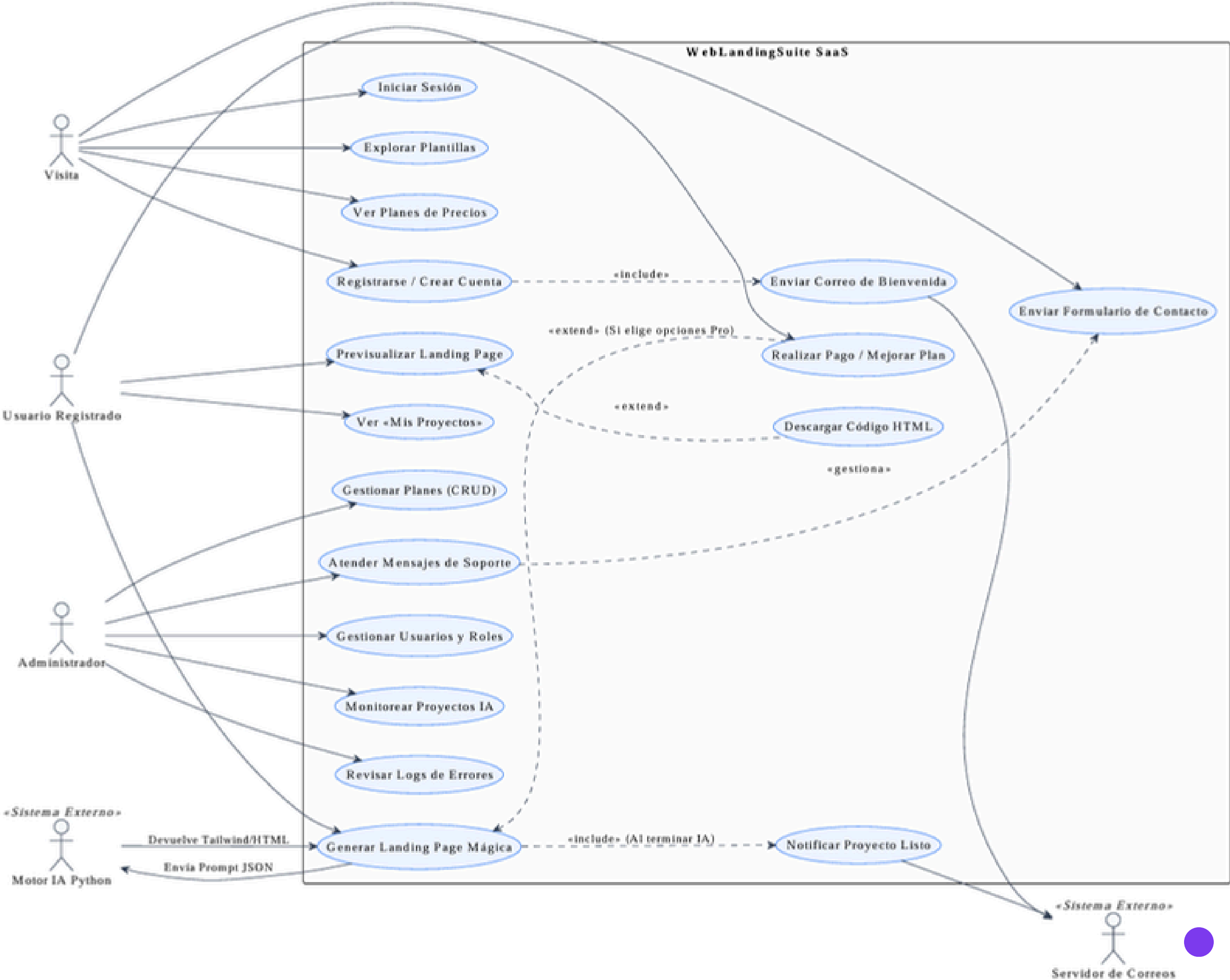
HU11: Como usuario, quiero visualizar en el Frontend el resultado final de mi Landing Page generada.

HU17: Como administrador, quiero visualizar el listado completo de usuarios registrados en la plataforma.

HU16: Como administrador, quiero consultar el historial completo de eventos (Logs) para detectar actividades sospechosas o errores.



Diagrama de Casos de Uso



Mockups del Sistema

IA

Landing

Beta v1.0

Mis Proyectos

Diseña tu Landing Page

Completa el formulario y nuestra IA maquetará el código por ti en tiempo real.

1

2

3

Nombre del Negocio

Clinica Vidaesdas

Selecciona tu Rubro

Salud y Bienestar

Consultoria

Tecnologia

Descripción Breve

Ofrecemos terapias integrales para mejorar tu calidad de vida.

Atras

Siguiente Paso →

https://landing.ia/preview/c3/clinica-vida

Clinica Vida

Inicio Servicios

Reservar

SALUD Y BIENESTAR

La solución inteligente para
tu bienestar integral.

Ofrecemos terapias integrales para mejorar tu calidad de vida con los mejores
especialistas del país.

Comenzar Ahora

Mockups del Sistema

IA

Landing

← Volver al Editor

Elige tu plan y descarga el código.

Pago único. Sin suscripciones mensuales. Eres dueño del 100% de los archivos para alojarlos donde quieras.

RECOMENDADO

Plan Básico

Ideal para validación rápida de tu idea.

\$0

/ Gratis

✓ HTML5/CSS3 Estático

✓ Estructura simple (1 sección)

✗ Sin animaciones complejas

Descargar Gratis (.zip)

Plan Intermedio

El estándar para negocios serios y profesionales.

\$19.99

/ pago único

✓ Diseño Totalmente Responsivo

✓ Tailwind CSS Integrado

✓ 3 Secciones + Formulario

Seleccionar Plan →

Plan Premium

Para agencias, consultores y alto rendimiento.

\$49.99

/ pago único

✓ Animaciones Avanzadas (Scroll)

✓ Optimización SEO Técnica

✓ Soporte Prioritario de Despliegue

Seleccionar Plan

Pago Seguro

Todas las transacciones están encriptadas. No guardamos los datos de tu tarjeta.

INFORMACIÓN DE LA TARJETA

0000 0000 0000 0000

MM/AA

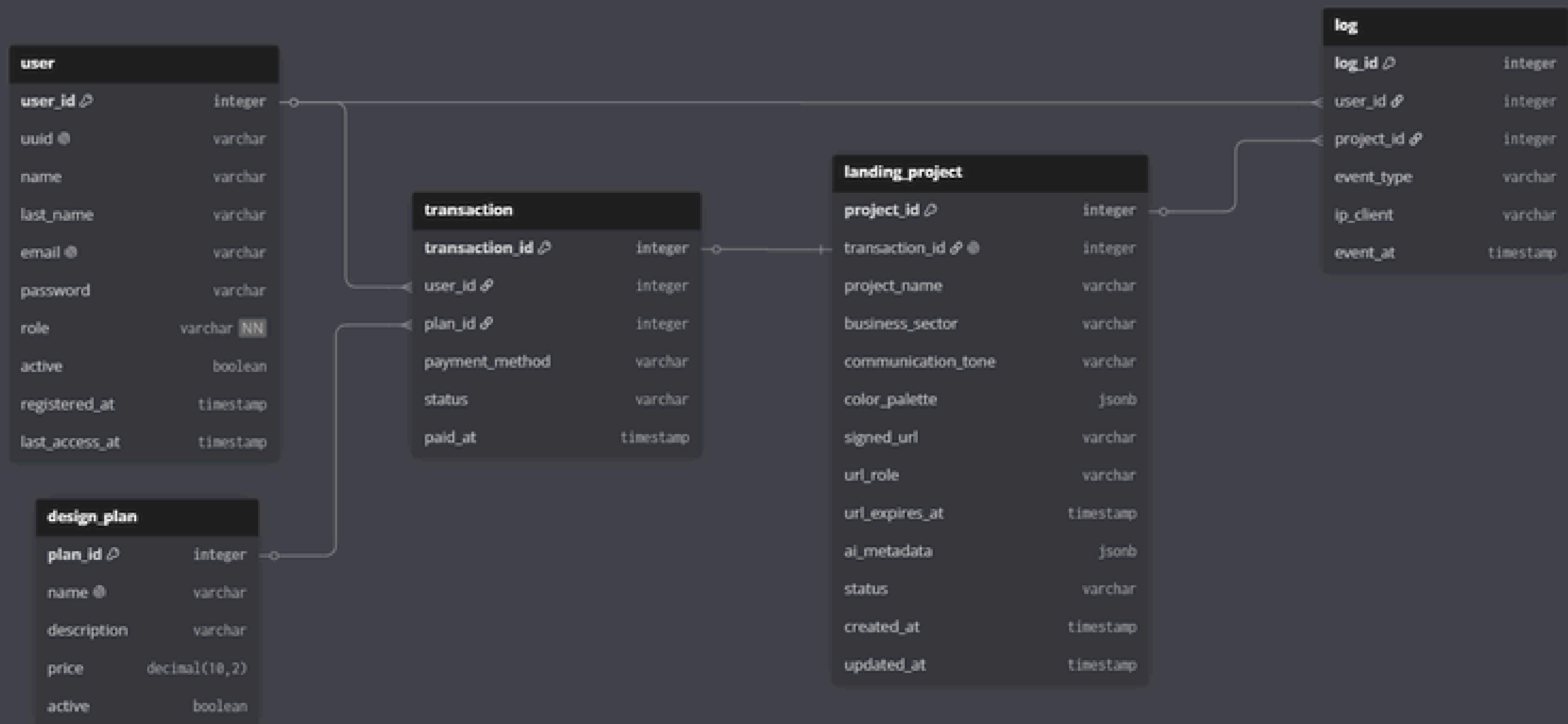
CVC

Tu código está empaquetado

Recibirás un archivo .zip con HTML5, CSS3, e imágenes optimizadas listas para desplegar en tu hosting.

Descargar Sitio (.ZIP)

Modelo de Datos (DER)



Arquitectura de Software y Patrones de Diseño

Frontend — React 19 SPA

Atomic Design: átomos / moléculas / organismos / páginas
React Router DOM · Controlled Components · Tailwind CSS

Backend Central — Spring Boot 3.2.5 (Java 21)

Arquitectura 3 capas: Controller → Service → Repository
Patrones: DTO, Builder (Lombok), Intercepting Filter (JWT)

Microservicio IA — FastAPI (Python 3)

Desacoplado del backend central. Recibe payload JSON con prompt estructurado y retorna HTML/CSS al orquestador (Spring Boot).

Base de Datos — PostgreSQL (Neon)

ORM con Spring Data JPA / Hibernate. ACID garantizado.
Atributo jsonb para metadatos dinámicos de la IA.

Patrones Aplicados

Atomic Design (UI)

Átomos → Moléculas → Organismos → Páginas

MVC / 3 Capas

Controller / Service / Repository en Spring Boot

DTO Pattern

UserRequestDTO, AiResponseDTO — valida y oculta entidades

Builder (Lombok)

@Builder para instanciación fluida de entidades

Intercepting Filter

JwtAuthenticationFilter intercepta cada request HTTP

Microservicio

IA desacoplada en Python/FastAPI, comunicación HTTP JSON

Repository Pattern

Spring Data JPA abstrae el acceso a PostgreSQL

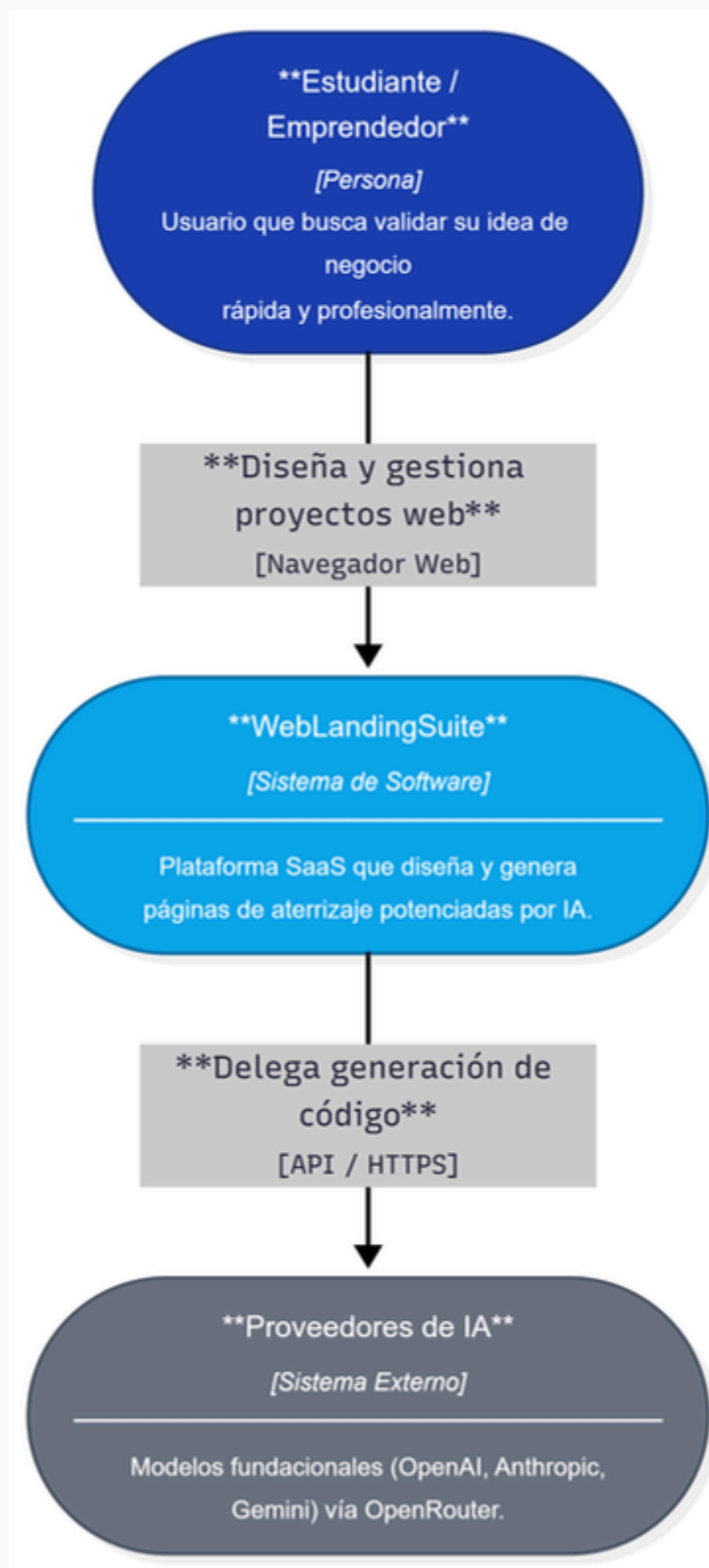
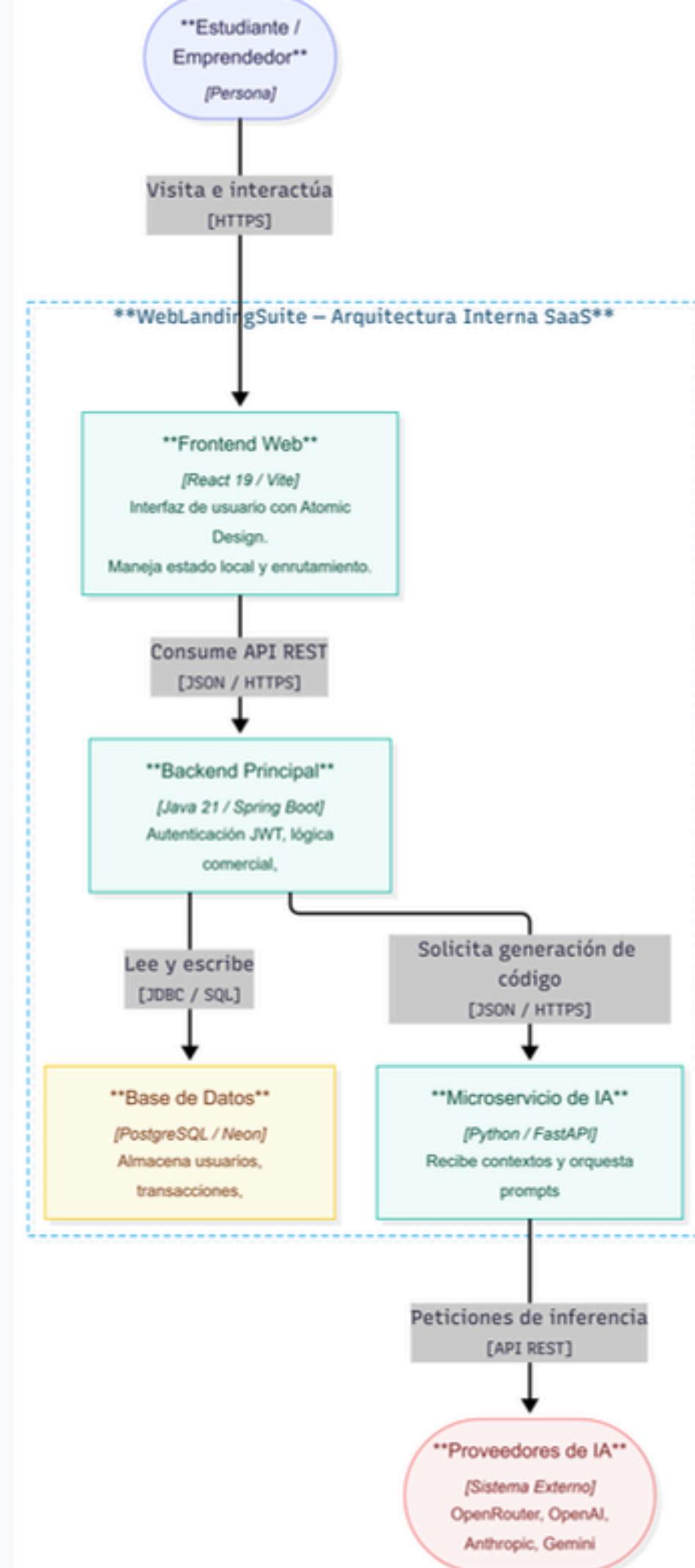
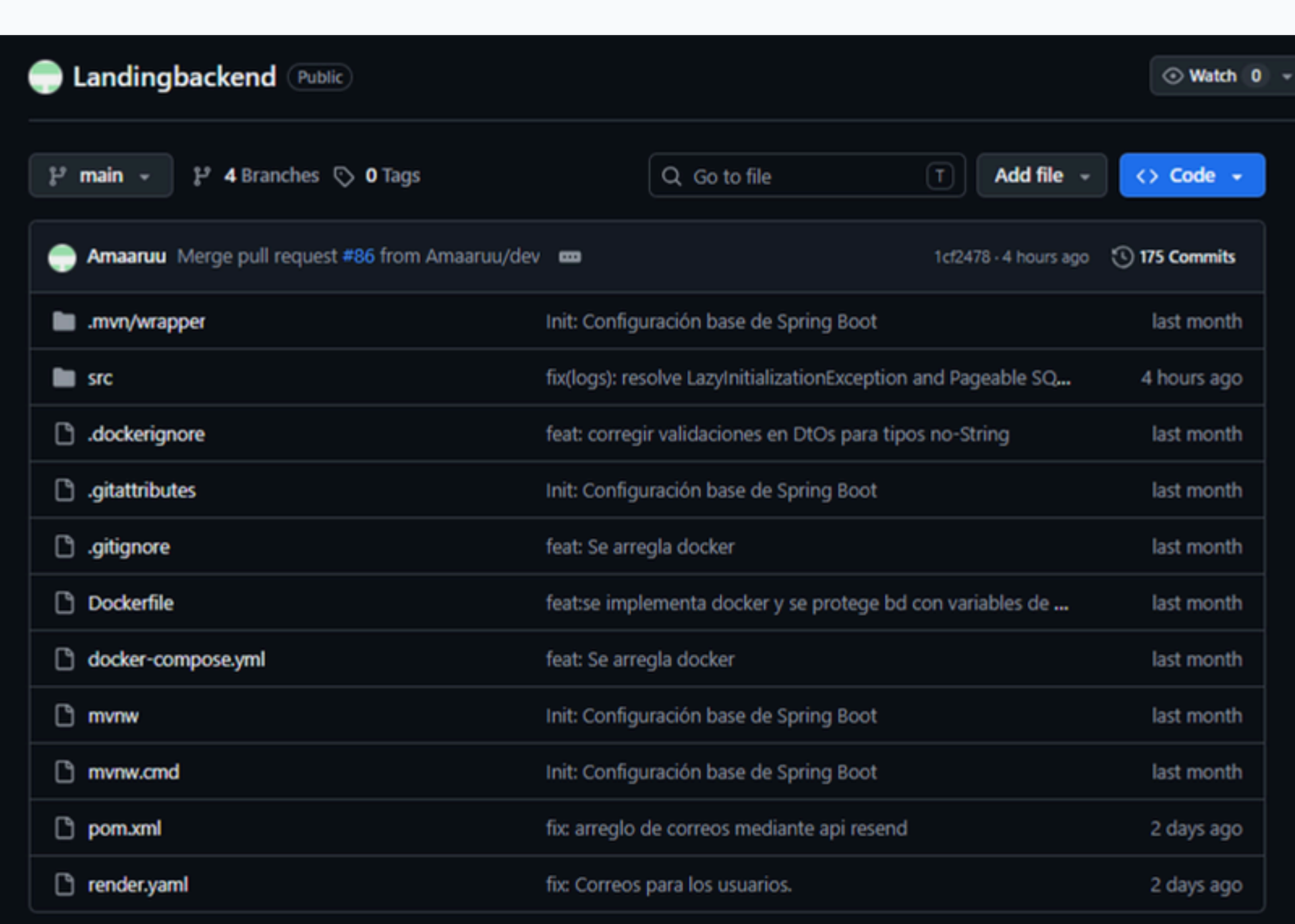


Diagrama de Contenedores C4 / Nivel 1 - 2

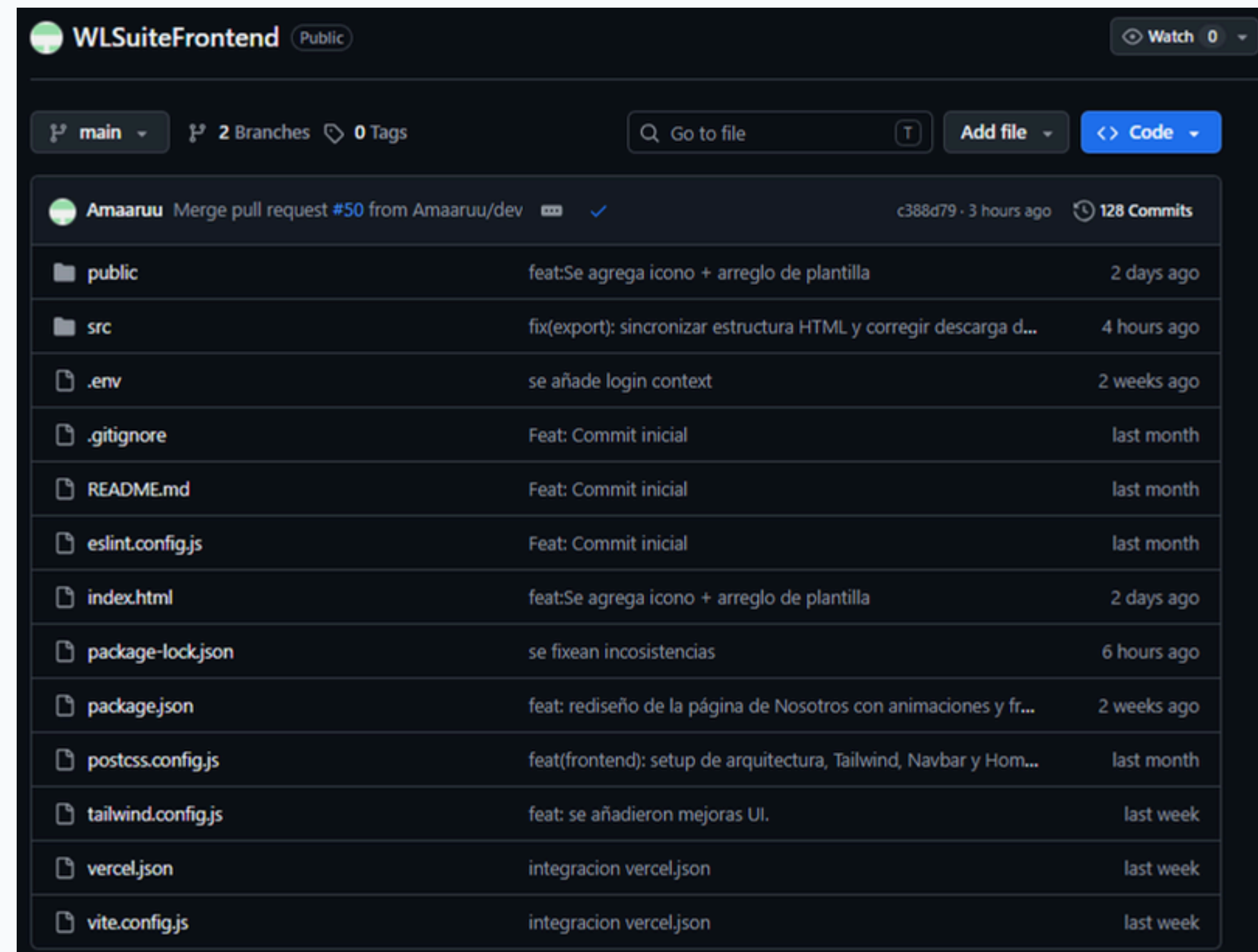


Stack Tecnológico Definitivo

Frontend	Backend (Java)	Microservicio IA (Python)
React 19 SPA con React Router DOM v7.14.2 Vite 8 Bundler + HMR optimizado Tailwind CSS v3.4 Paleta 'sapphire' + glassmorphism Axios v1.15.2 Cliente HTTP para consumo de API Lucide React Iconografía v1.11.0 ESLint v9 herramienta de análisis estático de código diseñada para encontrar y solucionar problemas en proyectos de JavaScript y TypeScript	Java 21 Runtime JDK — Spring Boot 3.2.5 Spring Security + JWT 0.11.5 Autenticación stateless JWT Spring Data JPA + Hibernate ORM para PostgreSQL (Neon) Springdoc OpenAPI 2.5.0 Swagger — documentación interactiva Lombok @Data, @Builder, @Slf4j — reduce boilerplate jakarta.validation Validación de DTOs con anotaciones	Python 3 + FastAPI Asíncrono, ideal para LLMs Pydantic Tipado estricto de payloads entrantes OpenAI SDK Integración con OpenRouter (Gemini / GPT / Claude) Docker Contenerización del microservicio



<https://github.com/Amaaruu/Landingbackend.git>



<https://github.com/Amaaruu/WLSuiteFrontend.git>



 WLSuitePythonAPI Public

Watch 0

main

1 Branch

0 Tags

Add file

<> Code

 **SOCRAM53** fix: correccion de modelos

258f113 · 2 days ago 🕒 18 Commits

 .gitignore	feat: primer commit del motor IA con OpenRouter	last month
 main.py	fix: correccion de modelos	2 days ago
 requirements.txt	feat: primer commit del motor IA con OpenRouter	last month

<https://github.com/Amaaruu/WLSuitePythonAPI.git>

Commits on May 22, 2026

fix: correccion de modelos

 SOCRAM53 committed 2 days ago

258f113  

Commits on May 20, 2026

se arreglan json

 SOCRAM53 committed 3 days ago

affe2a1  

Commits on May 19, 2026

Merge branch 'main' of <https://github.com/Amaaruu/WLSuitePythonAPI>

 SOCRAM53 committed 4 days ago

6a90410  

Adaptaciones

 SOCRAM53 committed 4 days ago

ebb8fe1  

fix: devuelta a cambios antiguos.

 Juan committed 4 days ago

7cf3ba9  

Revert "actualizacion de landings" 

 Juan committed 4 days ago

b30d8fb  

actualizacion de landings

 Juan committed 4 days ago

06f9400  



Arquitecturas del Backend y Frontend + Pull Request

LANDINGBACKEND

> .github

> .mvn

> .vscode

> src

main

java \ Landing \ Backend

config

controller

dto

exception

model

repository

security

service

BackendApplication.java

resources

application.properties

> test

> target

.dockerignore

.env

.gitattributes

.gitignore

docker-compose.yml

Dockerfile

mvnw

mvnw.cmd

pom.xml

render.yaml

WLSUITEFRONTEND

> node_modules

> public

> src

assets

components

atoms

molecules

organisms

config

context

hooks

pages

services

utils

App.jsx

index.css

main.jsx

.env

.gitignore

eslint.config.js

index.html

package-lock.json

package.json

postcss.config.js

README.md

tailwind.config.js

vercel.json

vite.config.js

Commits on May 23, 2026

Merge pull request #86 from Amaaruu/dev

Amaaruu authored 4 hours ago

Verified1cf2478

Merge pull request #85 from Amaaruu/feature/fix-logs

Amaaruu authored 4 hours ago

Verified148034b

fix(logs): resolve LazyInitializationException and Pageable SQL collision

Amaaruu committed 4 hours ago

16c816b

feat: se arreglen logs

Amaaruu committed 4 hours ago

005a4de

Commits on May 22, 2026

Merge pull request #84 from Amaaruu/dev

Amaaruu authored yesterday

Verified004acbb

fix: Correos para los usuarios.

Juan committed 2 days ago

55a51ef

Commits on May 23, 2026

Merge pull request #50 from Amaaruu/dev

Amaaruu authored 4 hours ago · 1 / 1

Verifiedc388d79

Merge pull request #49 from Amaaruu/feature/IGS-fix-export

Amaaruu authored 4 hours ago · 2 / 2

Verified15cfb0c

fix(export): sincronizar estructura HTML y corregir descarga de assets en exportProject

Amaaruu committed 4 hours ago · 2 / 2

26b1085

fix(export): unify export logic with viewer and fix UI components

Amaaruu committed 4 hours ago · 2 / 2

92c7bed

Merge pull request #48 from Amaaruu/dev

Amaaruu authored 6 hours ago · 1 / 1

Verifiedada432f

fix js

Amaaruu committed 6 hours ago · 2 / 2

a1bcd59

se fixean incosistencias

Amaaruu committed 6 hours ago · 2 / 2

5ba9c9a

Merge pull request #47 from Amaaruu/feature/IGS-fix-templates

Amaaruu authored 6 hours ago · 2 / 2

Verifieda7a3cd5

Configuración de Servidor de Producción

01	Plataforma Cloud y Componentes Desplegados • Frontend (SPA): Desplegado en Vercel (Aprovisionamiento automático global mediante Edge Network). • Backend (API REST): Desplegado en Render (Servicio web administrado con auto-scaling básico). • Servicio de IA (Python): Desplegado en Render como microservicio independiente comunicado de forma asíncrona. • Base de Datos Relacional: Hospedada en Neon DB (Arquitectura PostgreSQL Serverless sobre AWS).	02	Runtime, Versiones y Puertos • Entornos de Ejecución: Java 21 (OpenJDK) / Spring Boot 3.2.5 — Node.js 20+ / React 19. • Motor de Base de Datos: PostgreSQL 16. Puertos y Direccionamiento: • Desarrollo local: Backend en puerto 8080, Frontend en puerto 5173. • Producción: Asignación dinámica del puerto mediante la variable de entorno <code>PORT</code> de Render.
03	Monitoreo y Acceso Público • Trazabilidad de Errores: Centralización de logs vía consola nativa de Render (Live Logs) combinado con persistencia interna de excepciones críticas en la entidad Log de la Base de Datos. • Cifrado de Extremo a Extremo: Protocolo seguro HTTPS mediante certificados SSL/TLS automáticos gestionados por las plataformas Vercel y Render.	04	Pipeline CI/CD, Build y Arranque • Automatización Continuo (CI/CD): Pipeline conectado automáticamente vía Webhooks a los repositorios de GitHub (ramas main). • Comandos de Construcción (Build Stage): • Backend (Maven): <code>./mvnw clean package -DskipTests</code> (Generación de empaquetado ejecutable <code>.jar</code>). • Frontend (NPM): <code>npm run build</code> (Compilación estática optimizada en directorio <code>/dist</code>). • Comando de Arranque (Start): <code>java -jar target/classes/...</code> (Invocado automáticamente por el Service Runner de Render).



Deploy Render Backend

My Workspace

Environment

Landingbackend

Events

Settings

MONITOR

Logs

Metrics

MANAGE

Changelog

Invite a friend

Contact support

Render Status

My project / Production / Landingbackend

Search

+ New Upgrade

All logs Search

May 23, 4:02 PM - 4:08 PM GMT-4

May 23

04:07:18 PM [2fkh0] 2026-05-23T20:07:18.148Z INFO 1 --- [LandingBackend] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 10000 (http) with context path ''

04:07:18 PM [2fkh0] 2026-05-23T20:07:18.148Z INFO 1 --- [LandingBackend] [main] Landing.Backend.BackendApplication : Started BackendApplication in 116.997 seconds (process running for 127.104)

04:07:19 PM [2fkh0] 2026-05-23T20:07:19.044Z INFO 1 --- [LandingBackend] [io-10000-exec-1] o.s.a.e.c.C.[Tomcat]. [localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'

04:07:19 PM [2fkh0] 2026-05-23T20:07:19.044Z INFO 1 --- [LandingBackend] [io-10000-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'

04:07:19 PM [2fkh0] 2026-05-23T20:07:19.046Z INFO 1 --- [LandingBackend] [io-10000-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms

04:07:28 PM --> Your service is live

04:07:28 PM -->

04:07:28 PM --> //

04:07:28 PM -->

04:07:28 PM --> Available at your primary URL https://landingbackend-sirk.onrender.com

04:07:28 PM -->

04:07:28 PM --> //

Need better ways to work with logs? Try the Render CLI, Render MCP Server, or set up a log stream integration



Deploy Render API Python

My Workspace

Environment

WLSuitePythonAPI

Events

Settings

MONITOR

Logs

Metrics

MANAGE

Environment

Shell

Scaling

Previews

Disk

One-Off Jobs

Changelog

Invite a friend

Contact support

Render Status

My project / Production / WLSuitePythonAPI

Search

+ New

Upgrade

Connect

Manual Deploy

WLSuitePythonAPI Python 3 Free Upgrade your instance

Service ID: srv-d88ged8jp88kc73ck88g

Amaarui / WLSuitePythonAPI main

https://wlsuitepythonapi-bib6.onrender.com

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. Upgrade now

May 22, 2026 at 12:13 AM Live

2581113 fix correccion de modelos

All logs Search May 22, 12:12 AM - 12:15 AM GMT-4

May 22

12:13:41 AM ==> Deploying...

12:13:41 AM ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance

12:13:51 AM [7xk9c] ==> Running 'uvicorn main:app --host 0.0.0.0 --port \$PORT'

12:14:11 AM [7xk9c] INFO: Started server process [58]

12:14:11 AM [7xk9c] INFO: Waiting for application startup.

12:14:11 AM [7xk9c] INFO: Application startup complete.

12:14:11 AM [7xk9c] INFO: Uvicorn running on http://0.0.0.0:10000 (Press CTRL-C to quit)

12:14:11 AM [7xk9c] INFO: 127.0.0.1:51564 - "HEAD / HTTP/1.1" 405 Method Not Allowed

12:14:12 AM ==> Your service is live

12:14:13 AM ==>

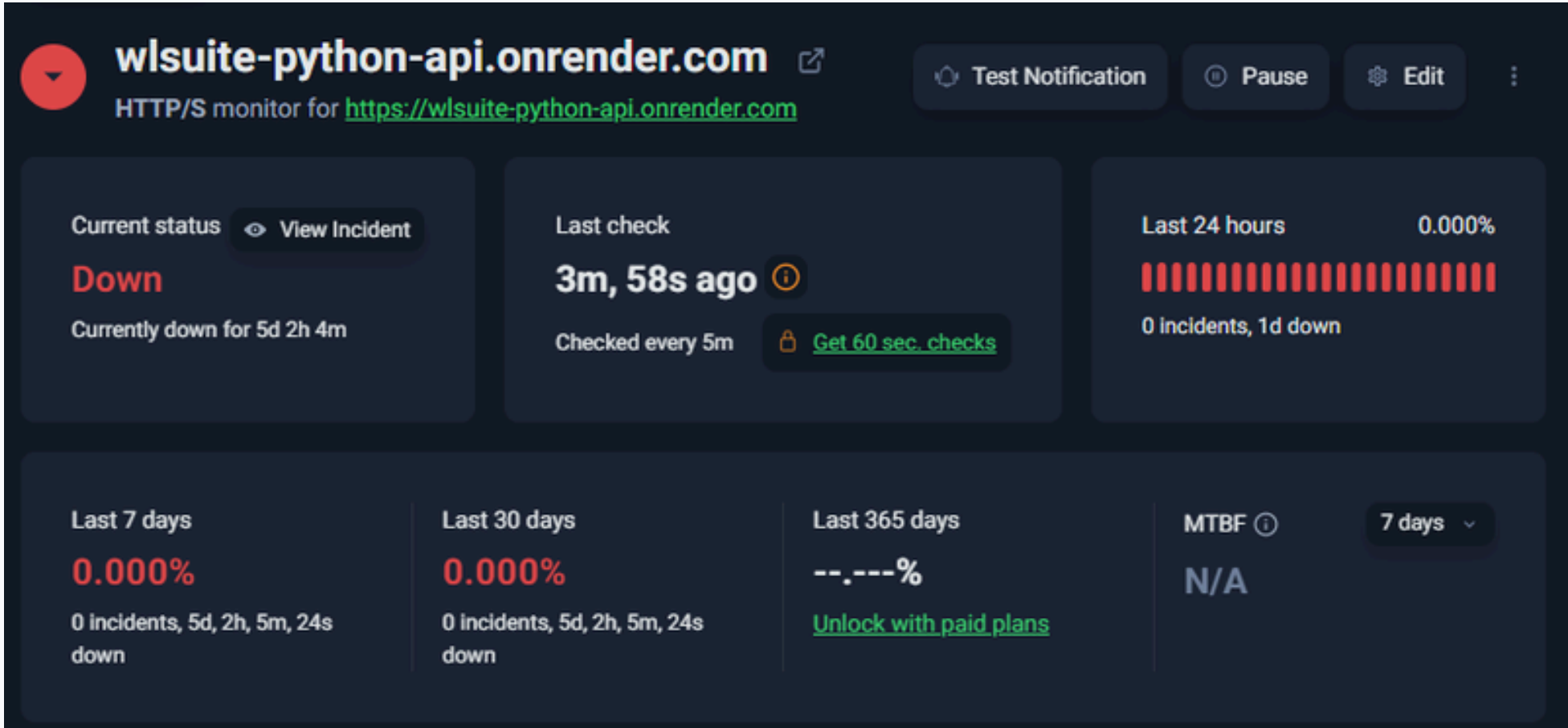
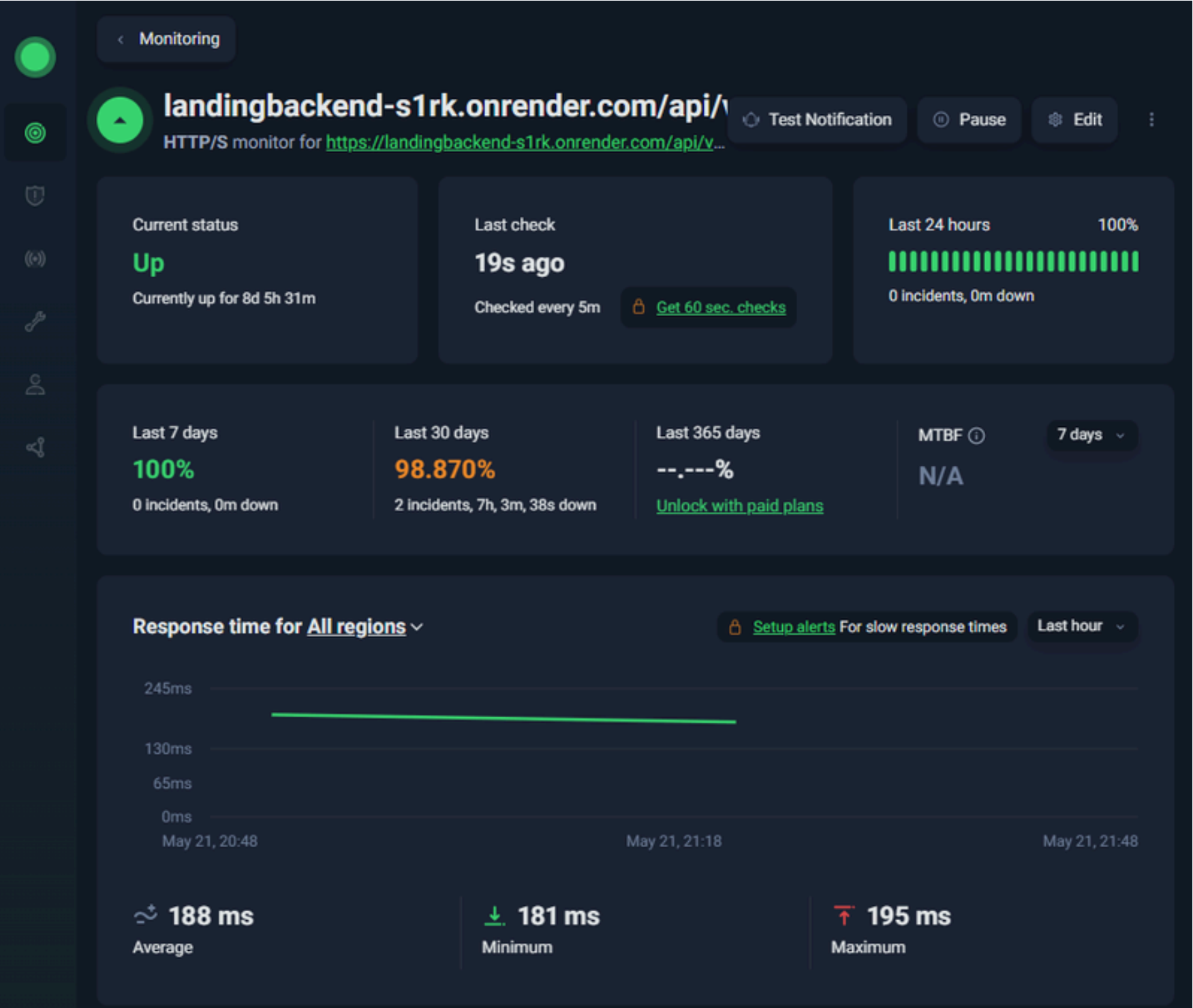
12:14:13 AM ==> Available at your primary URL https://wlsuitepythonapi-bib6.onrender.com

12:14:13 AM ==>

12:14:13 AM [7xk9c] INFO: 34.89.77.94:0 - "GET / HTTP/1.1" 200 OK

Need better ways to work with logs? Try the Render CLI, Render MCP Server, or set up a log stream integration

Deploy UptimeRobot



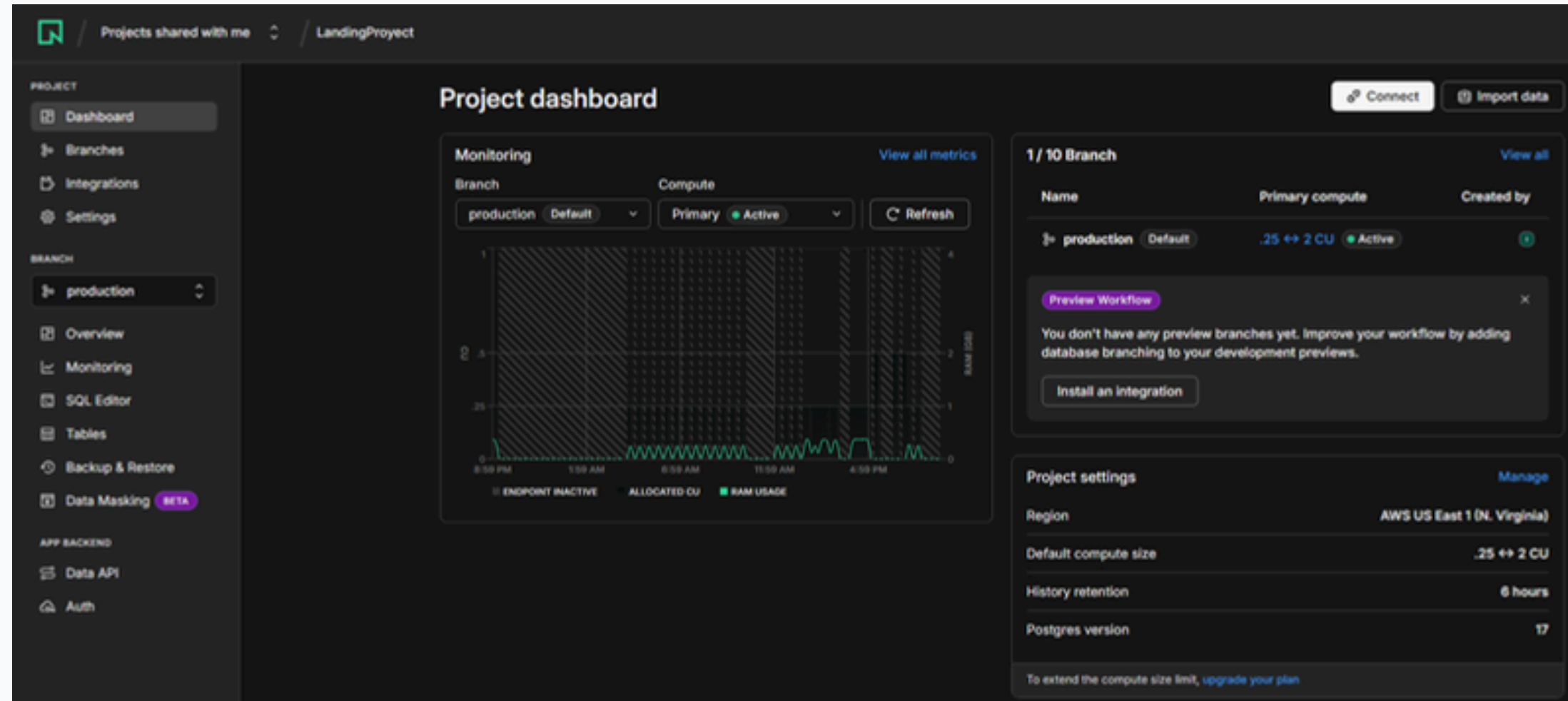
Deploy vercel Frontend

wl-suite-frontend ↕		Deployments		...
🔍 All Bran...	🔍 All Auth...	All Environments	📅 Select Date Range	🌈 Status 5/6
BMygWm2Z3 Production 🔄 Current		● Ready 6s	🔗 main 🔗 c388d79 Merge pull request #50 fr...	4h ago by Amaaruu  ...
2GR71br8S Preview		● Ready 6s	🔗 dev 🔗 15cfb0c Merge pull request #49 fr...	4h ago by Amaaruu  ...
AT7xZt3Gj Preview		● Ready 9s	🔗 feature/IGS-fix-export 🔗 26b1085 fix(export): sincronizar estru...	4h ago by Amaaruu  ...
Cerq6xb3U Preview		● Ready 9s	🔗 feature/IGS-fix-export 🔗 92c7bed fix(export): unify export logic...	4h ago by Amaaruu  ...
C92Zhg8g8 Production ⓘ		● Ready 11s	🔗 main 🔗 ada432f Merge pull request #48 fr...	6h ago by Amaaruu  ...
B5Gfqclag Preview		● Ready 8s	🔗 dev 🔗 a1bcd59 fix js	6h ago by Amaaruu  ...

fd4298626e8060383c0e5



Base de Datos



The screenshot displays the 'Tables' interface for the 'production' branch. It shows a list of tables: neondb, public, design_plan, landing_project, log, transaction, and users. The 'users' table is selected, and its structure is shown in the 'DATA' tab. The table has 9 columns: active, user_id, registered_at, email, last_name, name, password, and a primary key column.

active	user_id	registered_at	email	last_name	name	password
TRUE	1	2020-05-23 17:00:21.09...	isaac.gonzalez@gmail...	Gonzalez	Isaac	\$2a\$10\$g1zC83Xc8RX...
FALSE	2	2020-05-23 18:35:29.08...	n814133883@gmail.com	Gonzalez	Nicolas	\$2a\$10\$1qYA004rguq...
TRUE	3	2020-05-23 18:37:16.38...	n814133883@gmail.com	Gonzalez	Nicolas	\$2a\$10\$4/cx7b7ygdX...
TRUE	4	2020-05-23 19:13:07.58...	juankagmail.com	Agüero, Carcano	Juan Carlos	\$2a\$10\$X33x08xje88...
TRUE	5	2020-05-23 22:31:01.14...	orellanamarco329@gmail...	Orellana	Marco	\$2a\$10\$2yYk89c20xx...
TRUE	6	2020-05-24 01:03:20.54...	weblandingsuite@gmail...	Testing	Joan	\$2a\$10\$WpajcPaaAt...
TRUE	7	2020-05-25 00:42:48.05...	juankaguero370@gmail...	Agüero, Carcano	Juanka	\$2a\$10\$0xP88u3Gawp...
TRUE	8	2020-05-25 01:39:09.15...	nicolasbarker13@gmail...	Barker	Nicolas	\$2a\$10\$M0qe.pl876e...

URL de Producción, Acceso a la App y Demo en vivo

APLICACIÓN EN PRODUCCIÓN

www.weblandingsuite.com

Credenciales de prueba

Usuario: weblandingsuite@gmail.com

Contraseña: 12345678

Rol: Admin

Admin: juanka@gmail.com

contraseña: lokillo124

Repositorios

Frontend: Amaaruu/WLSuiteFrontend

Backend: Amaaruu/Landingbackend

IA API: Amaaruu/WLSuitePythonAPI

Rama deploy: main (protegida)

Paneles Cloud Abiertos

Frontend: Vercel Dashboard

Backend/IA: Render Dashboard

Base de Datos: Neon Console

Monitoreo: UptimeRobot

Matriz de Avance de Funcionalidades

#	Funcionalidad	Descripción breve	Evidencia	Estado
1	Autenticación JWT	Registro, login, logout, token stateless	URL + PR mergeado	IMPLEMENTADA
2	Formulario Inteligente	Captura de reqs. en < 10 pasos con validación en tiempo real	URL + PR mergeado	IMPLEMENTADA
3	Motor Prompt Engineering	3 niveles (Básico/Intermedio/Premium) → OpenRouter	URL + PR mergeado	IMPLEMENTADA
4	Post-proceso + Live Preview	Sanitiza HTML/CSS, renderiza en iframe aislado	URL + PR mergeado	IMPLEMENTADA
5	Descarga .zip	Empaqueta HTML/CSS/JS listos para hosting	URL + demostración	IMPLEMENTADA
6	Dashboard de Proyectos	Historial de landings generadas por usuario	URL + PR mergeado	IMPLEMENTADA
7	Auditoría / Logs	Registro de eventos críticos con IP y timestamp	URL + PR mergeado	IMPLEMENTADA
8	Gestión de Usuarios (Admin)	Panel admin para listar y gestionar usuarios	URL + PR mergeado	IMPLEMENTADA
9	Notificaciones / Correo SMTP	Avisos automáticos al usuario (bienvenida, descarga)	URL + PR mergeado	IMPLEMENTADA

Integraciones Necesarias — Servicios Externos

SaaS — Motor IA

OpenRouter / OpenAI API

Entrada: Prompt estructurado JSON (nivel + datos del cliente)

Salida: Código HTML/CSS/JS de la landing page

Seguridad: API Key en variable de entorno OPENROUTER_API_KEY (nunca en código)

DBaaS — Persistencia

Neon (PostgreSQL DBaaS)

Entrada: Queries SQL desde Spring Boot (JPA)

Salida: Registros de users, transactions, landing_projects, logs

Seguridad: DB_URL con credenciales en vars de entorno de Render

Resend — API Correos

Entrada: Payload JSON (Destinatario, asunto, cuerpo en texto/HTML, reply-to).

Salida: Status 200 OK / 403 Forbidden, ID único de mensaje enviado.

Seguridad: API Key inyectada como variable de entorno en Render (RESEND_API_KEY).

PaaS — CI/CD Frontend

Vercel Webhooks

Entrada: Push/merge a rama main en GitHub

Salida: Build automático y despliegue de la SPA en CDN global

Seguridad: Token de integración GitHub-Vercel gestionado en el panel

SaaS — Monitoreo

UptimeRobot

Entrada: Ping HTTP a endpoints /health del backend y microservicio IA

Salida: Alertas email/Slack ante caídas. Dashboard de uptime histórico

Seguridad: Sin credenciales sensibles; monitorea URLs públicas

Cloudinary — Media Manager

Entrada: Archivo multimedia (MultipartFile) desde el Frontend.

Salida: URL segura (HTTPS), metadatos de la imagen (formato, peso, dimensiones).

Seguridad: API Secret y API Key gestionados en el servidor Backend; firma de peticiones (Signed Uploads).



Conclusión — WebLandingSuite

Problema → Solución Demostrada

Microempresarios y freelancers ya pueden generar landing pages profesionales en minutos, llenando un formulario, sin saber programar y con pago único.

MVP en Producción

Frontend (Vercel) + Backend Spring Boot (Render) + Microservicio FastAPI (Render) + PostgreSQL (Neon) operativos y públicamente accesibles.

Beneficios Validados

Generación IA en 3 niveles funcional · JWT + auditoría activos · CI/CD automático desde GitHub · Descarga .zip operativa · Dashboard de proyectos.

Lecciones del Despliegue

Gestión de secretos con vars de entorno · Configuración CORS · Validación de prompts para evitar alucinaciones IA · Git Flow con Code Review obligatorio.



Anexos

Carta Gantt:

<https://docs.google.com/spreadsheets/d/1iY7XXo1Mn2V9pTDYK2O2z5PMAXIOYwQc7IAM7dUQiyI/edit?usp=sharing>

Jira Landing:

<https://duocuc-team-z4zvnc8.atlassian.net/jira/software/projects/SCRUM/boards/1>

Repositorio LandingBackend:

<https://github.com/Amaaruu/Landingbackend.git>

Repositorio WLSuiteFrontend:

<https://github.com/Amaaruu/WLSuiteFrontend.git>

Repositorio WLSuitePythonAPI:

<https://github.com/Amaaruu/WLSuitePythonAPI.git>

Base de datos:

<https://console.neon.tech/app/projects/empty-block-19817322>

Diagramas:

<https://drive.google.com/drive/folders/1pL1Shjxxfea5Md7HiqVKvphVDvkeuDSn?usp=sharing>

